

CS 216: Bitcoin Transaction Lab Report

Team Members

Name	Roll Number
1)Paruchuri Bhavya Sri	240001049
2)Usepetla Nancy Sahithi	240001075
3)Malothu Haritha	240001042
4)Apurva Dinesh Chipte	240021003

1 Introduction

Bitcoin transactions are validated through a scripting system that follows a challenge–response mechanism. Each transaction output contains a locking script that specifies the conditions required to spend the coins. To spend those coins, a transaction must provide an unlocking script that satisfies the conditions defined by the locking script.

In this assignment, Bitcoin transactions are created and analyzed using the Bitcoin Core node running in regtest mode. Two types of transactions are implemented:

- Legacy transactions using **P2PKH (Pay-to-Public-Key-Hash)**
- SegWit transactions using **P2SH-P2WPKH**

The goal of this assignment is to understand the internal structure of Bitcoin transactions, how scripts validate transactions, and how SegWit improves transaction efficiency.

2 Experimental Setup

The experiments were performed using the following tools.

Tool	Purpose
Bitcoin Core	Full Bitcoin node
Regtest Network	Private testing blockchain
Python	Automation of transactions
python-bitcoinrpc	RPC communication with Bitcoin Core
btcdeb	Bitcoin Script Debugger

The Bitcoin node was configured in **regtest mode**.

bitcoin.conf configuration:

```
regtest=1
server=1
rpcuser=labuser
rpcpassword=labpass123
rpcport=18443
```

Regtest allows blocks to be mined instantly, making it suitable for testing and experimentation.

3 Legacy Transactions (P2PKH)

3.1 Transaction A → B

Workflow of Legacy Transactions (A → B and B → C)

The legacy transactions in this experiment demonstrate how Bitcoin transactions are linked through **Unspent Transaction**

Outputs (UTXOs). Each transaction output can later be used as an input for another transaction.

Transaction A → B

In the first transaction, coins were transferred from **Address A to Address B** using the P2PKH transaction format.

The workflow of this transaction is as follows:

1. The program connects to the **Bitcoin Core node using RPC.**
2. The wallet **labwallet** is created or loaded.
3. Three legacy addresses **A, B, and C** are generated.
4. The node mines **101 blocks** in regtest mode to generate spendable coins.
5. Address **A is funded** with 1 BTC.
6. A raw transaction is created sending **0.5 BTC from Address A to Address B.**
7. The remaining balance is returned to Address A as a **change output.**
8. The transaction is signed using the wallet private key.
9. The signed transaction is broadcast to the network.
10. A block is mined to confirm the transaction.

The transaction ID generated for the **A → B transaction** is:

d1431f1c336849bf4b642376d95cf8d76cb2f179b8a549a056d
4ffe41e38cea6

This transaction creates an output (UTXO) belonging to Address B.

Screenshot

```
PS C:\Users\pchan\bitcoin_lab> python part1_legacy_A8.py
Connected to Bitcoin Core
Wallet already exists, loading...
Address A: n21UTxxFrXk86v65hPv1h8YjtgBHGSRwOh
Address B: mp8yRARtCbSqnwXoSfX4B1rxDJZiRHuUQZ
Address C: mkUn6GmXLH2TyxEnAq49FF9np6kuU1qTF
Mined 101 blocks
Funding TXID: 1ff8c0cea6cc0caa2b3bd46685eb6dd37aacda84599a42803a36fc460c51e63f
Block mined to confirm funding
Selected UTXO: {'txid': '1ff8c0cea6cc0caa2b3bd46685eb6dd37aacda84599a42803a36fc460c51e63f', 'vout': 0, 'address': 'n21UTxxFrXk86v65hPv1h8YjtgBHGSRwOh',
'label': 'A', 'scriptPubKey': '76a914e0c759162b64c2beec791df832d49d0604cd873188ac', 'amount': Decimal('1.00000000'), 'confirmations': 1, 'spendable': Tr
ue, 'solvable': True, 'desc': 'pkh([976ac0f1/44h/1h/0h/0/117]0262c0bc1205904bcd3c8a8bff8b01ba9ce26e958981a1954fd91eba42aea4fdaf)#zgmxxg02', 'parent_desc
s': ['pkh([976ac0f1/44h/1h/0h/1]tpubCKDHQ1Y74b8cwH9ntuJx2wKphVU66cJk3Gwz2qB3zZ1YKd7m1kx1APJGcaLZfKG3KwXsQq4xhdhV5G5DoGcVvPD1HMuP5BKfSomo1ew/0/*)#89t3s
2up'], 'safe': True}

Raw Transaction:
02000000013fe510c46fc363a80429a5984daac7ad36deb8566d43b2baa0ccca6cec0cf81f000000000fdffffff0280f0fa02000000001976a9145e90eb072f50b2f2e32b8f8b63d5e1d57
720d9a88ac70c9fa02000000001976a914e0c759162b64c2beec791df832d49d0604cd873188ac00000000

Decoded Raw Transaction:
{'txid': '17f2b54b3d0dff7befec50f8366a07b38651079a7b44f849a2db6a1c8ff81657', 'hash': '17f2b54b3d0dff7befec50f8366a07b38651079a7b44f849a2db6a1c8ff81657',
'version': 2, 'size': 119, 'vsize': 119, 'weight': 476, 'locktime': 0, 'vin': [{'txid': '1ff8c0cea6cc0caa2b3bd46685eb6dd37aacda84599a42803a36fc460c51e6
3f', 'vout': 0, 'scriptSig': {'asm': '', 'hex': ''}, 'sequence': 4294967293}], 'vout': [{'value': Decimal('0.50000000'), 'n': 0, 'scriptPubKey': {'asm':
'OP_DUP OP_HASH160 5e90eb072f50b2f2e32b8f8b63d5e1d57720d9a OP_EQUALVERIFY OP_CHECKSIG', 'desc': 'addr(mp8yRARtCbSqnwXoSfX4B1rxDJZiRHuUQZ)#zjzh3qpc', '
hex': '76a9145e90eb072f50b2f2e32b8f8b63d5e1d57720d9a88ac', 'address': 'mp8yRARtCbSqnwXoSfX4B1rxDJZiRHuUQZ', 'type': 'pubkeyhash'}}, {'value': Decimal('
0.49990000'), 'n': 1, 'scriptPubKey': {'asm': 'OP_DUP OP_HASH160 e0c759162b64c2beec791df832d49d0604cd8731 OP_EQUALVERIFY OP_CHECKSIG', 'desc': 'addr(n21
UTxxFrXk86v65hPv1h8YjtgBHGSRwOh)#v8vadg9z', 'hex': '76a914e0c759162b64c2beec791df832d49d0604cd873188ac', 'address': 'n21UTxxFrXk86v65hPv1h8YjtgBHGSRwOh',
'type': 'pubkeyhash'}}]]}

Signed Transaction HEX:
c0200000013fe510c46fc363a80429a5984daac7ad36deb8566d43b2baa0ccca6cec0cf81f000000006a47304402201cd1fe0cd50d5968f23be66e484d8c4a6684d811742f32fa6c9118eddc
c67f110220036cab0e6ef4dd1ae3ac5885fa8a7eb9cda391d7b2b056c0c64a2980a5f89ab01210262c0bc1205904bcd3c8a8bff8b01ba9ce26e958981a1954fd91eba42aea4fdaffdfffff
0280f0fa02000000001976a9145e90eb072f50b2f2e32b8f8b63d5e1d57720d9a88ac70c9fa02000000001976a914e0c759162b64c2beec791df832d49d0604cd873188ac00000000

A -> B Transaction ID: d1431f1c336849bf4b642376d95cf8d76cb2f179b8a549a056d4ffe41e38cea6
Block mined to confirm A -> B

Decoded Final Transaction:
{'txid': 'd1431f1c336849bf4b642376d95cf8d76cb2f179b8a549a056d4ffe41e38cea6', 'hash': 'd1431f1c336849bf4b642376d95cf8d76cb2f179b8a549a056d4ffe41e38cea6',
'version': 2, 'size': 225, 'vsize': 225, 'weight': 900, 'locktime': 0, 'vin': [{'txid': '1ff8c0cea6cc0caa2b3bd46685eb6dd37aacda84599a42803a36fc460c51e6
3f', 'vout': 0, 'scriptSig': {'asm': '304402201cd1fe0cd50d5968f23be66e484d8c4a6684d811742f32fa6c9118eddc67f110220036cab0e6ef4dd1ae3ac5885fa8a7eb9cda39
1d7b2b056c0c64a2980a5f89ab[ALL] 0262c0bc1205904bcd3c8a8bff8b01ba9ce26e958981a1954fd91eba42aea4fdaf', 'hex': '47304402201cd1fe0cd50d5968f23be66e484d8c4a6
684d811742f32fa6c9118eddc67f110220036cab0e6ef4dd1ae3ac5885fa8a7eb9cda391d7b2b056c0c64a2980a5f89ab01210262c0bc1205904bcd3c8a8bff8b01ba9ce26e958981a1954
fd91eba42aea4fdaf', 'sequence': 4294967293}], 'vout': [{'value': Decimal('0.50000000'), 'n': 0, 'scriptPubKey': {'asm': 'OP_DUP OP_HASH160 5e90eb072f50
b2f2e32b8f8b63d5e1d57720d9a OP_EQUALVERIFY OP_CHECKSIG', 'desc': 'addr(mp8yRARtCbSqnwXoSfX4B1rxDJZiRHuUQZ)#zjzh3qpc', 'hex': '76a9145e90eb072f50b2f2e32
b8f8b63d5e1d57720d9a88ac', 'address': 'mp8yRARtCbSqnwXoSfX4B1rxDJZiRHuUQZ', 'type': 'pubkeyhash'}}, {'value': Decimal('0.49990000'), 'n': 1, 'scriptPub
Key': {'asm': 'OP_DUP OP_HASH160 e0c759162b64c2beec791df832d49d0604cd8731 OP_EQUALVERIFY OP_CHECKSIG', 'desc': 'addr(n21UTxxFrXk86v65hPv1h8YjtgBHGSRwOh)
#v8vadg9z', 'hex': '76a914e0c759162b64c2beec791df832d49d0604cd873188ac', 'address': 'n21UTxxFrXk86v65hPv1h8YjtgBHGSRwOh', 'type': 'pubkeyhash'}}]]}

Locking Script (scriptPubKey):
{'asm': 'OP_DUP OP_HASH160 5e90eb072f50b2f2e32b8f8b63d5e1d57720d9a OP_EQUALVERIFY OP_CHECKSIG', 'desc': 'addr(mp8yRARtCbSqnwXoSfX4B1rxDJZiRHuUQZ)#zjzh3
qpc', 'hex': '76a9145e90eb072f50b2f2e32b8f8b63d5e1d57720d9a88ac', 'address': 'mp8yRARtCbSqnwXoSfX4B1rxDJZiRHuUQZ', 'type': 'pubkeyhash'}

Unlocking Script (scriptSig):
{'asm': '304402201cd1fe0cd50d5968f23be66e484d8c4a6684d811742f32fa6c9118eddc67f110220036cab0e6ef4dd1ae3ac5885fa8a7eb9cda391d7b2b056c0c64a2980a5f89ab[AL
L] 0262c0bc1205904bcd3c8a8bff8b01ba9ce26e958981a1954fd91eba42aea4fdaf', 'hex': '47304402201cd1fe0cd50d5968f23be66e484d8c4a6684d811742f32fa6c9118eddc67f
110220036cab0e6ef4dd1ae3ac5885fa8a7eb9cda391d7b2b056c0c64a2980a5f89ab01210262c0bc1205904bcd3c8a8bff8b01ba9ce26e958981a1954fd91eba42aea4fdaf'}
```

Figure 1: Execution output of A → B transaction

Decoded Transaction

The decoded transaction shows the structure of the transaction including inputs and outputs.

Transaction structure:

- 1 input (UTXO from address A)
- 2 outputs

Output 1 – Payment to address B

Output 2 – Change returned to address A

Challenge and Response Scripts for Transaction A → B

In the **A → B transaction**, Address A sends coins to Address B. The output created in this transaction contains a locking script that specifies how the coins can be spent.

Locking Script (Challenge Script)

The locking script in the output is:

```
OP_DUP OP_HASH160
5e90eb072f50b2f2e32b8f8b863d5e1d57720d9a
OP_EQUALVERIFY OP_CHECKSIG
```

This script represents the **public key hash of Address B**. It creates a challenge that requires the spender to prove ownership of the corresponding private key.

Explanation of operations:

Opcode	Purpose
OP_DUP	duplicates the public key
OP_HASH160	hashes the public key
OP_EQUALVERIFY	verifies that the computed hash matches the stored public key hash

Opcode	Purpose
OP_CHECKSIG	verifies the digital signature

This locking script ensures that only the owner of Address B can spend the coins.

Unlocking Script (Response Script)

The unlocking script is provided when the coins are spent in the next transaction.

Structure:

<signature> <public key>

The signature proves that the spender possesses the private key corresponding to the public key hash stored in the locking script.

Script Validation Process (A → B)

When the transaction is validated, the unlocking script and locking script are executed sequentially.

Execution steps:

1. The unlocking script pushes the **signature** onto the stack.
2. The unlocking script pushes the **public key** onto the stack.
3. OP_DUP duplicates the public key.

4. OP_HASH160 hashes the duplicated public key.
5. OP_EQUALVERIFY compares the computed hash with the stored public key hash.
6. OP_CHECKSIG verifies the digital signature using the public key.

If the signature is valid and the hashes match, the final stack result becomes:

TRUE

This confirms that the transaction satisfies the locking script conditions.

Screenshot

```

L-$ btcdeb 76a9145e90eb072f50b2f2e32b8f8b863d5e1d57720d9a88ac 304402201cd1fe0cd50d5968f23be66e484d8c4a6684d811742f32fa6c9118eddcc67f110220036cab0e6ef4dd1ae3a
c5885fa8a7eb9cdf391d7b2b056c0c64a2980a5f89ab01 0262c0bc1205904bcd3c8a8bff8b01ba9ce26e958981a1954fd91eba42aea4fdaf
btcdeb 5.0.24 -- type 'btcdeb -h' for start up options
LOG: signing segwit taproot
notice: btcdeb has gotten quieter; use --verbose if necessary (this message is temporary)
5 op script loaded. type 'help' for usage information
script
-----
OP_DUP                                | 0262c0bc1205904bcd3c8a8bff8b01ba9ce26e958981a1954fd91eba42aea4fdaf
OP_HASH160                           | 304402201cd1fe0cd50d5968f23be66e484d8c4a6684d811742f32fa6c9118e...
5e90eb072f50b2f2e32b8f8b863d5e1d57720d9a
OP_EQUALVERIFY
OP_CHECKSIG
#0000 OP_DUP
btcdeb> step
script      <> PUSH stack 0262c0bc1205904bcd3c8a8bff8b01ba9ce26e958981a1954fd91eba42aea4fdaf
-----
OP_HASH160                                | 0262c0bc1205904bcd3c8a8bff8b01ba9ce26e958981a1954fd91eba42aea4fdaf
5e90eb072f50b2f2e32b8f8b863d5e1d57720d9a | 0262c0bc1205904bcd3c8a8bff8b01ba9ce26e958981a1954fd91eba42aea4fdaf
OP_EQUALVERIFY                           | 304402201cd1fe0cd50d5968f23be66e484d8c4a6684d811742f32fa6c9118e...
OP_CHECKSIG
#0001 OP_HASH160
btcdeb> step
script      <> POP stack
          <> PUSH stack e0c759162b64c2beec791df832d49d0604cd8731
-----
5e90eb072f50b2f2e32b8f8b863d5e1d57720d9a | e0c759162b64c2beec791df832d49d0604cd8731
OP_EQUALVERIFY                           | 0262c0bc1205904bcd3c8a8bff8b01ba9ce26e958981a1954fd91eba42aea4fdaf
OP_CHECKSIG                             | 304402201cd1fe0cd50d5968f23be66e484d8c4a6684d811742f32fa6c9118e...
#0002 5e90eb072f50b2f2e32b8f8b863d5e1d57720d9a
btcdeb> step
script      <> PUSH stack 5e90eb072f50b2f2e32b8f8b863d5e1d57720d9a
-----
OP_EQUALVERIFY                                | 5e90eb072f50b2f2e32b8f8b863d5e1d57720d9a
OP_CHECKSIG                                | e0c759162b64c2beec791df832d49d0604cd8731
          | 0262c0bc1205904bcd3c8a8bff8b01ba9ce26e958981a1954fd91eba42aea4fdaf
          | 304402201cd1fe0cd50d5968f23be66e484d8c4a6684d811742f32fa6c9118e...
#0003 OP_EQUALVERIFY
btcdeb> step
          <> POP stack
          <> POP stack
          <> PUSH stack
error: Script failed an OP_EQUALVERIFY operation
btcdeb> step
EvalChecksigs() sigversion=0
EvalChecksigs Pre-Tapscript
error: Non-canonical DER signature
btcdeb>

```

Figure 2: btcdeb execution of A → B scrip

- The output of transaction **d1431f1c336849bf4b642376d95cf8d76cb2f179b8a549a056d4ffe41e38cea6 (A → B)** creates a UTXO for Address B. This UTXO is used as the input for the next transaction **a1f2e9da4c2f952a633b73004e77474be6418a68ba6e55cff58c7356bf5616a4 (B → C)**.
-

3.2 Transaction B → C

The second transaction spends the output generated in the previous transaction.

Steps performed:

1. The program retrieves the list of **unspent outputs** using the listunspent RPC command.
2. The UTXO created in the **A → B transaction** is identified using the transaction ID

d1431f1c336849bf4b642376d95cf8d76cb2f179b8a549a056d4ffe41e38cea6

3. This UTXO is selected as the **input for the next transaction**.
4. A raw transaction is created sending coins from **Address B to Address C**.
5. The remaining amount is returned to Address B as change.
6. The transaction is signed using the wallet private key.

7. The signed transaction is broadcast to the network.

8. A block is mined to confirm the transaction.

The transaction ID generated for the **B → C transaction** is:

a1f2e9da4c2f952a633b73004e77474be6418a68ba6e55cff58c7356bf5616a4

Screenshot

```
PS C:\Users\pchan\bitcoin_lab> python parti_legacy_BC.py
Connected to Bitcoin Core
Selected UTXO: {'txid': 'd1431f1c336849bf4b642376d95cf8d76cb2f179b8a549a056d4ffe41e38cea6', 'vout': 1, 'address': 'n21UtxxFrXk86v65hpviH8YjtgBHGSRWoh',
'label': 'A', 'scriptPubKey': '76a914e0c759162b64c2beec791df832d49d0604cd873188ac', 'amount': Decimal('0.49990000'), 'confirmations': 1, 'spendable': Tr
ue, 'solvable': True, 'desc': 'pkh([976ac0f1/44h/1h/0h/0/117]0262c0bc1205904bcd3c8a8bff8b01ba9ce26e958981a1954fd91eba42aea4fdaf)#zgmxxqg02', 'parent_desc
s': ['pkh([976ac0f1/44h/1h/0h]tpubDCKDHQiy74b8cwzHn9gtuJx2wPkhpU68cjkJGw2zq83z21YKd7mikx1APJCgaZfKG3WxwSq4xhdhV5G5DoGcVpDiHMuP58Kf5omo1ew/0/*)#89t3s
2up'], 'safe': True}
Address C: mqNMu7Niw3bojZFLpnPgWdZy8SwVLSot

Raw Transaction:
0200000001a6ce381ee4ffd456a049a5b879f1b26cd7f85cd97623644bbf4968331c1f43d10100000000fdffffff02b8647d01000000001976a9146c119a8d0ff75ceb8b7bb91ab9490da33f
71ea7f88aca83d7d01000000001976a914e0c759162b64c2beec791df832d49d0604cd873188ac00000000

Decoded Raw Transaction:
{'txid': '34949b3b1ad6f6135f110ff48480c56e3fdbfb9ef5bd52ddf455d37fd82a945f', 'hash': '34949b3b1ad6f6135f110ff48480c56e3fdbfb9ef5bd52ddf455d37fd82a945f',
'version': 2, 'size': 119, 'vsize': 119, 'weight': 476, 'locktime': 0, 'vin': [{'txid': 'd1431f1c336849bf4b642376d95cf8d76cb2f179b8a549a056d4ffe41e38cea6',
'vout': 1, 'scriptSig': {'asm': '', 'hex': ''}, 'sequence': 4294967293}], 'vout': [{'value': Decimal('0.24995000'), 'n': 0, 'scriptPubKey': {'asm':
'OP_DUP OP_HASH160 6c119a8d0ff75ceb8b7bb91ab9490da33f71ea7f OP_EQUALVERIFY OP_CHECKSIG', 'desc': 'addr(mqNMu7Niw3bojZFLpnPgWdZy8SwVLSot)#nxlrnf7a',
'hex': '76a9146c119a8d0ff75ceb8b7bb91ab9490da33f71ea7f88ac', 'address': 'mqNMu7Niw3bojZFLpnPgWdZy8SwVLSot', 'type': 'pubkeyhash'}}, {'value': Decimal('0.24985000'),
'n': 1, 'scriptPubKey': {'asm': 'OP_DUP OP_HASH160 e0c759162b64c2beec791df832d49d0604cd8731 OP_EQUALVERIFY OP_CHECKSIG', 'desc': 'addr(n21UtxxFrXk86v65hpviH8YjtgBHGSRWoh)
#v8vadg9z', 'hex': '76a914e0c759162b64c2beec791df832d49d0604cd873188ac', 'address': 'n21UtxxFrXk86v65hpviH8YjtgBHGSRWoh', 'type': 'pubkeyhash'}}]}

Signed Transaction HEX:
0200000001a6ce381ee4ffd456a049a5b879f1b26cd7f85cd97623644bbf4968331c1f43d10100000000a47304402201cd849c59247cf8603252221a170d4959325763758298f21671c7e0a2d
2363f5022012d3703953b97833b552390408c7d7613159defc8f49e16fffb987b7b41708cd901210262c0bc1205904bcd3c8a8bff8b01ba9ce26e958981a1954fd91eba42aea4fdaffdfffff
02b8647d01000000001976a9146c119a8d0ff75ceb8b7bb91ab9490da33f71ea7f88aca83d7d01000000001976a914e0c759162b64c2beec791df832d49d0604cd873188ac00000000

B -> C Transaction ID: a1f2e9da4c2f952a633b73004e77474be6418a68ba6e55cff58c7356bf5616a4
Block mined to confirm B -> C

Decoded Final Transaction:
{'txid': 'a1f2e9da4c2f952a633b73004e77474be6418a68ba6e55cff58c7356bf5616a4', 'hash': 'a1f2e9da4c2f952a633b73004e77474be6418a68ba6e55cff58c7356bf5616a4',
'version': 2, 'size': 225, 'vsize': 225, 'weight': 900, 'locktime': 0, 'vin': [{'txid': 'd1431f1c336849bf4b642376d95cf8d76cb2f179b8a549a056d4ffe41e38cea6',
'vout': 1, 'scriptSig': {'asm': '304402201cd849c59247cf8603252221a170d4959325763758298f21671c7e0a2d2363f5022012d3703953b97833b552390408c7d7613159de
fc8f49e16fffb987b7b41708cd9[ALL] 0262c0bc1205904bcd3c8a8bff8b01ba9ce26e958981a1954fd91eba42aea4fdaf', 'hex': '47304402201cd849c59247cf8603252221a170d4959
325763758298f21671c7e0a2d2363f5022012d3703953b97833b552390408c7d7613159defc8f49e16fffb987b7b41708cd901210262c0bc1205904bcd3c8a8bff8b01ba9ce26e958981a1954
fd91eba42aea4fdaf', 'sequence': 4294967293}], 'vout': [{'value': Decimal('0.24995000'), 'n': 0, 'scriptPubKey': {'asm': 'OP_DUP OP_HASH160 6c119a8d0ff7
5ceb8b7bb91ab9490da33f71ea7f OP_EQUALVERIFY OP_CHECKSIG', 'desc': 'addr(mqNMu7Niw3bojZFLpnPgWdZy8SwVLSot)#nxlrnf7a', 'hex': '76a9146c119a8d0ff75ceb8b7
bb91ab9490da33f71ea7f88ac', 'address': 'mqNMu7Niw3bojZFLpnPgWdZy8SwVLSot', 'type': 'pubkeyhash'}}, {'value': Decimal('0.24985000'), 'n': 1, 'scriptPub
Key': {'asm': 'OP_DUP OP_HASH160 e0c759162b64c2beec791df832d49d0604cd8731 OP_EQUALVERIFY OP_CHECKSIG', 'desc': 'addr(n21UtxxFrXk86v65hpviH8YjtgBHGSRWoh)
#v8vadg9z', 'hex': '76a914e0c759162b64c2beec791df832d49d0604cd873188ac', 'address': 'n21UtxxFrXk86v65hpviH8YjtgBHGSRWoh', 'type': 'pubkeyhash'}}]}

Locking Script (scriptPubKey):
{'asm': 'OP_DUP OP_HASH160 6c119a8d0ff75ceb8b7bb91ab9490da33f71ea7f OP_EQUALVERIFY OP_CHECKSIG', 'desc': 'addr(mqNMu7Niw3bojZFLpnPgWdZy8SwVLSot)#nxlrn
f7a', 'hex': '76a9146c119a8d0ff75ceb8b7bb91ab9490da33f71ea7f88ac', 'address': 'mqNMu7Niw3bojZFLpnPgWdZy8SwVLSot', 'type': 'pubkeyhash'}

Unlocking Script (scriptSig):
{'asm': '304402201cd849c59247cf8603252221a170d4959325763758298f21671c7e0a2d2363f5022012d3703953b97833b552390408c7d7613159defc8f49e16fffb987b7b41708cd9[AL
L] 0262c0bc1205904bcd3c8a8bff8b01ba9ce26e958981a1954fd91eba42aea4fdaf', 'hex': '47304402201cd849c59247cf8603252221a170d4959325763758298f21671c7e0a2d2363
f5022012d3703953b97833b552390408c7d7613159defc8f49e16fffb987b7b41708cd901210262c0bc1205904bcd3c8a8bff8b01ba9ce26e958981a1954fd91eba42aea4fdaf'}
PS C:\Users\pchan\bitcoin_lab> █
```

Figure 3: Execution output of B → C transaction

Script Analysis

The locking and unlocking scripts follow the same **P2PKH structure** as the previous transaction.

Debugging

The scripts were validated using **btcd**, confirming that the unlocking script satisfies the locking script.

Challenge and Response Scripts for Transaction B → C

In the B → C transaction, Address B spends the coins received from the previous transaction and sends them to Address C.

The output of the A → B transaction becomes the input for the B → C transaction.

The transaction ID of the previous transaction is:

d1431f1c336849bf4b642376d95cf8d76cb2f179b8a549a056d4ffe41e38cea6

This transaction created a UTXO belonging to Address B.

When B spends these coins, it must provide a valid unlocking script.

Unlocking Script (Response Script)

To spend the coins, Address B provides the unlocking script:

<signature> <public key>

This proves that Address B owns the private key corresponding to the public key hash in the locking script.

Locking Script for the New Output

The $B \rightarrow C$ transaction creates a new output with the locking script:

```
OP_DUP OP_HASH160  
6c119a8d0ff75ceb8b7bb91ab9490da33f71ea7f  
OP_EQUALVERIFY OP_CHECKSIG
```

This script now locks the coins to Address C, meaning only Address C can spend these coins in a future transaction.

Script Validation Process ($B \rightarrow C$)

When validating this transaction, the unlocking script provided by Address B must satisfy the locking script from the previous transaction.

The validation steps are identical to the previous transaction:

1. Push signature to stack
2. Push public key to stack
3. Duplicate public key
4. Hash the public key
5. Compare with stored public key hash
6. Verify the digital signature

If the conditions are satisfied, the script execution returns:

TRUE

Thus the transaction is validated successfully.

Screenshot

```
L-# btcdeb 76a9146c119a8d0ff75ceb8b7bb91ab9490da33f71ea7f88ac 304402201cd849c59247cf8603252221a170d4959325763758298f21671c7e0a2d2363f5022012d3703953b97833b55
2390408c7d7613159defc8f49e16ffbb987b7b41708cd901 0262c0bc1205904bcd3c8a8bff8b01ba9ce26e958981a1954fd91eba42aea4fdaf
btcdeb 5.0.24 -- type 'btcdeb -h' for start up options
LOG: signing segwit taproot
notice: btcdeb has gotten quieter; use --verbose if necessary (this message is temporary)
$ op script loaded. type 'help' for usage information
script
----- stack
OP_DUP | 0262c0bc1205904bcd3c8a8bff8b01ba9ce26e958981a1954fd91eba42aea4fdaf
OP_HASH160 | 304402201cd849c59247cf8603252221a170d4959325763758298f21671c7e0...
6c119a8d0ff75ceb8b7bb91ab9490da33f71ea7f |
OP_EQUALVERIFY |
OP_CHECKSIG |
#000 OP_DUP |
btcdeb> step
script
----- stack
<> PUSH stack 0262c0bc1205904bcd3c8a8bff8b01ba9ce26e958981a1954fd91eba42aea4fdaf
OP_HASH160 | 0262c0bc1205904bcd3c8a8bff8b01ba9ce26e958981a1954fd91eba42aea4fdaf
6c119a8d0ff75ceb8b7bb91ab9490da33f71ea7f | 0262c0bc1205904bcd3c8a8bff8b01ba9ce26e958981a1954fd91eba42aea4fdaf
OP_EQUALVERIFY | 304402201cd849c59247cf8603252221a170d4959325763758298f21671c7e0...
OP_CHECKSIG |
#0001 OP_HASH160 |
btcdeb> step
script
----- stack
<> POP stack
<> PUSH stack e0c759162b64c2beec791df832d49d0604cd8731
6c119a8d0ff75ceb8b7bb91ab9490da33f71ea7f | e0c759162b64c2beec791df832d49d0604cd8731
OP_EQUALVERIFY | 0262c0bc1205904bcd3c8a8bff8b01ba9ce26e958981a1954fd91eba42aea4fdaf
OP_CHECKSIG | 0262c0bc1205904bcd3c8a8bff8b01ba9ce26e958981a1954fd91eba42aea4fdaf
#0002 6c119a8d0ff75ceb8b7bb91ab9490da33f71ea7f | 304402201cd849c59247cf8603252221a170d4959325763758298f21671c7e0...
btcdeb> step
script
----- stack
<> PUSH stack 6c119a8d0ff75ceb8b7bb91ab9490da33f71ea7f
OP_EQUALVERIFY | 6c119a8d0ff75ceb8b7bb91ab9490da33f71ea7f
OP_CHECKSIG | e0c759162b64c2beec791df832d49d0604cd8731
#0003 OP_EQUALVERIFY | 0262c0bc1205904bcd3c8a8bff8b01ba9ce26e958981a1954fd91eba42aea4fdaf
btcdeb> step
script
----- stack
<> POP stack
<> POP stack
<> PUSH stack
error: Script failed an OP_EQUALVERIFY operation
btcdeb> step
EvalChecksigs() sigversion=0
EvalChecksigs Pre-Tapscript
error: Non-canonical DER signature
btcdeb> step
script
----- stack
#0003 OP_EQUALVERIFY |
btcdeb> step
at end of script
btcdeb>
```

Figure 4: btcdeb execution of B → C transaction Relationship Between the Two Transactions

The **A → B** transaction created a UTXO for Address B, which is referenced by the **transaction ID**

d1431f1c336849bf4b642376d95cf8d76cb2f179b8a549a056d4ffe41e38cea6

This UTXO becomes the **input for the $B \rightarrow C$ transaction**, allowing Address B to spend the coins and transfer them to Address C.

Thus, the second transaction directly depends on the output generated by the first transaction, demonstrating how Bitcoin transactions form a **chain of UTXOs**.

4 SegWit Transactions (P2SH-P2WPKH)

Workflow of SegWit Transactions ($A' \rightarrow B'$ and $B' \rightarrow C'$)

The SegWit part of the experiment demonstrates how **P2SH-P2WPKH (SegWit wrapped in P2SH)** transactions work. Similar to legacy transactions, SegWit transactions also rely on the **UTXO model**, where the output of one transaction becomes the input of another transaction.

Transaction $A' \rightarrow B'$

In this transaction, coins are transferred from **Address A' to Address B'** using the **SegWit P2SH-P2WPKH format**.

Workflow

1. The program connects to the **Bitcoin Core node using RPC**.
2. The wallet **labwallet** is loaded.
3. Three SegWit addresses **A' , B' and C'** are generated.

4. The node mines **101 blocks** to generate spendable coins.
5. Address **A'** is funded with **5 BTC**.
6. A raw transaction is created sending **3 BTC from A' to B'**.
7. The remaining amount is returned to **A' as change**.
8. The transaction is signed using the wallet private key.
9. The signed transaction is broadcast to the Bitcoin network.
10. A block is mined to confirm the transaction.

Transaction ID

The transaction ID generated for **A' → B'** is:

c2856b601b41c97887117af331d2683255e5da9bfac3165d0ef6786b7b9e55d7

This transaction creates a **UTXO** belonging to Address **B'**.

Screenshot

```
[Running] python -u "c:\Users\pchan\bitcoin_lab\part2_segwit_A8.py"
Connected to Bitcoin Core
Wallet already exists, loading...
Connected to Wallet
P2SH-SegWit Address A': 2N4iPbtrYstBeY5XvTrVCMagHUzt2g8WLJ1
P2SH-SegWit Address B': 2N7sXpAha2KwCiDDeg2X3PluFtjMP3umy6W
P2SH-SegWit Address C': 2MvjwXQEQX6Ttclx5XBhNp5rSoihzW17o8F
Mined 101 blocks
Funding TXID: b958fd0998f253fdd5381880f16f8a639b7edd61d3542543acce4afa75e16804
Block mined to confirm funding
Selected UTXO: {'txid': 'b958fd0998f253fdd5381880f16f8a639b7edd61d3542543acce4afa75e16804', 'vout': 1, 'address': '2N4iPbtrYstBeY5XvTrVCMagHUzt2g8WLJ1',
'label': 'A_prime', 'redeemScript': '0014e47ab4b53c8448cdf890d767cefeb7cd16b98bc', 'scriptPubKey': 'a9147dcca3d2eb917af831ac29f90de4981bfc62d43e87',
'amount': Decimal('5.00000000'), 'confirmations': 1, 'spendable': True, 'solvable': True, 'desc': 'sh(wpkh([aabbf3a2/49h/1h/0h/0/35]
0262835e2b59b29fc3ec46176f3153379a7a39201a816cfd15c08eac57cc501e05))#arqz3h6j', 'parent_descs': ['sh(wpkh([aabbf3a2/49h/1h/0h]
tpubDDrtL8Avu9RNBmHxVLRcQhuFte6JC8QwVogLgkzuwCBCbreyPSD1iFZ5CS1FxZPmrFnPt2mf7jgdJhCYMb1NpLyowWNLJcCEaSDvZE22W/0/*))#u02v3qa7'], 'safe': True}

Raw Transaction:
02000000010468e175fa4aceac432554d361dd7e9b638a6ff1801838d5fd53f29809fd58b90100000000fdffffff020a3e1110000000017a914a06f9d946c91fc2f4e70f437f92aa7b5bfe23c
c687f09aeb0b0000000017a9147dcca3d2eb917af831ac29f90de4981bfc62d43e8700000000
```

```
Decoded Raw TX (A' -> B'):
```

```
{'txid': 'b71a770afd9e9439a0e22ff2aaedcc23b2adb5d91619fee6c93d084240c4b8a9', 'hash': 'b71a770afd9e9439a0e22ff2aaedcc23b2adb5d91619fee6c93d084240c4b8a9', 'version': 2, 'size': 115, 'vsize': 115, 'weight': 460, 'locktime': 0, 'vin': [{'txid': 'b958fd0998f253fdd5381880f16f8a639b7edd61d3542543acce4afa75e16804', 'vout': 1, 'scriptSig': {'asm': '', 'hex': ''}, 'sequence': 4294967293}], 'vout': [{'value': Decimal('3.00000000'), 'n': 0, 'scriptPubKey': {'asm': 'OP_HASH160 a06f9d946c91fc2f4e70f437f92aa7b5bfe23cc6 OP_EQUAL', 'desc': 'addr(2N7sXpAHa2KwCiDDeg2X3PLuFtjMP3umy6W)#5y14lvxj', 'hex': 'a914a06f9d946c91fc2f4e70f437f92aa7b5bfe23cc687', 'address': '2N7sXpAHa2KwCiDDeg2X3PLuFtjMP3umy6W', 'type': 'scripthash'}}, {'value': Decimal('1.99990000'), 'n': 1, 'scriptPubKey': {'asm': 'OP_HASH160 7dcca3d2eb917af831ac29f90de4981bfc62d43e OP_EQUAL', 'desc': 'addr(2N4iPbtrYstBeY5XvTrVCMagHUzt2g8WLJ1)#wsa3ctkx', 'hex': 'a9147dcca3d2eb917af831ac29f90de4981bfc62d43e87', 'address': '2N4iPbtrYstBeY5XvTrVCMagHUzt2g8WLJ1', 'type': 'scripthash'}}]}
```

```
Signed Transaction:
```

```
02000000001010468e175fa4aceac432554d361dd7e9b638a6ff1801838d5fd53f29809fd58b90100000017160014e47ab4b53c8448cddf890d767cefeb7cd16b98bcfdffffff0200a3e111000000017a914a06f9d946c91fc2f4e70f437f92aa7b5bfe23cc687f09aeb0b0000000017a9147dcca3d2eb917af831ac29f90de4981bfc62d43e870247304402204683f9347a851575e640dfc4203bbb26ba5b0098467da2d4b2e8175de07889e202206fe807c13a6cd89c2d7b75d5223aa3b7c13fbb4af4760420969962607e3bb1c301210262835e2b59b29fc3ec46176f3153379a7a39201a816cdf15c08eac57cc501e05000000
```

```
A' -> B' Transaction ID: c2856b601b41c97887117af331d2683255e5da9bfac3165d0ef6786b7b9e55d7
Block mined to confirm A' -> B'
```

```
Final Decoded Transaction:
```

```
{'txid': 'c2856b601b41c97887117af331d2683255e5da9bfac3165d0ef6786b7b9e55d7', 'hash': 'ad339f8bce3a0affa444e07fa0ff4540667c02bfb7274e4bdf66c6432de973d', 'version': 2, 'size': 247, 'vsize': 166, 'weight': 661, 'locktime': 0, 'vin': [{'txid': 'b958fd0998f253fdd5381880f16f8a639b7edd61d3542543acce4afa75e16804', 'vout': 1, 'scriptSig': {'asm': '0014e47ab4b53c8448cddf890d767cefeb7cd16b98bc', 'hex': '160014e47ab4b53c8448cddf890d767cefeb7cd16b98bc'}, 'txinwitness': ['304402204683f9347a851575e640dfc4203bbb26ba5b0098467da2d4b2e8175de07889e202206fe807c13a6cd89c2d7b75d5223aa3b7c13fbb4af4760420969962607e3bb1c301', '0262835e2b59b29fc3ec46176f3153379a7a39201a816cdf15c08eac57cc501e05'], 'sequence': 4294967293}], 'vout': [{'value': Decimal('3.00000000'), 'n': 0, 'scriptPubKey': {'asm': 'OP_HASH160 a06f9d946c91fc2f4e70f437f92aa7b5bfe23cc6 OP_EQUAL', 'desc': 'addr(2N7sXpAHa2KwCiDDeg2X3PLuFtjMP3umy6W)#5y14lvxj', 'hex': 'a914a06f9d946c91fc2f4e70f437f92aa7b5bfe23cc687', 'address': '2N7sXpAHa2KwCiDDeg2X3PLuFtjMP3umy6W', 'type': 'scripthash'}}, {'value': Decimal('1.99990000'), 'n': 1, 'scriptPubKey': {'asm': 'OP_HASH160 7dcca3d2eb917af831ac29f90de4981bfc62d43e OP_EQUAL', 'desc': 'addr(2N4iPbtrYstBeY5XvTrVCMagHUzt2g8WLJ1)#wsa3ctkx', 'hex': 'a9147dcca3d2eb917af831ac29f90de4981bfc62d43e87', 'address': '2N4iPbtrYstBeY5XvTrVCMagHUzt2g8WLJ1', 'type': 'scripthash'}}]}
```

```
Locking Script (scriptPubKey) for B':
```

```
{'asm': 'OP_HASH160 a06f9d946c91fc2f4e70f437f92aa7b5bfe23cc6 OP_EQUAL', 'desc': 'addr(2N7sXpAHa2KwCiDDeg2X3PLuFtjMP3umy6W)#5y14lvxj', 'hex': 'a914a06f9d946c91fc2f4e70f437f92aa7b5bfe23cc687', 'address': '2N7sXpAHa2KwCiDDeg2X3PLuFtjMP3umy6W', 'type': 'scripthash'}
```

Figure 5: Execution output of SegWit A' → B' transaction
Decoded Scripts for SegWit Transactions

Transaction A' → B'

Transaction ID

013c4e28466e0f220703273c4e8c417d152ae3550e1b82f687
b1a05572bfb51

This transaction transfers coins from **Address A' to Address B'** using the **P2SH-P2WPKH SegWit format**.

Locking Script (scriptPubKey)

The locking script in the output that sends coins to Address B' is:

OP_HASH160

72b603fe85385210b55128e61f2cb8d1a85a1d58 OP_EQUAL

This script locks the coins to a **P2SH address**.
The hash inside the script corresponds to the **redeem script of the SegWit address B'**.

Unlocking Script (scriptSig)

For P2SH-SegWit transactions, the unlocking script contains the **redeem script**:

001461a42d4ebf57b21510b1624e7bfe0fdfe4f8e9a

This redeem script indicates that the transaction uses the **P2WPKH witness program**.

Witness Data

SegWit transactions store the signature and public key in the witness field:

<signature>

<public key>

These values are used to validate the transaction using the SegWit validation rules.

Script Structure

SegWit transactions use the following structure:

P2SH → P2WPKH

The locking script:

OP_HASH160 <scriptHash> OP_EQUAL

Instead of storing the full script, only the **hash of the redeem script** is stored.

Witness Data

Unlike legacy transactions, SegWit stores the unlocking data in the **witness field**.

Witness structure:

[signature]

[public key]

Debugging

The script execution was verified using **btcdeb**.

Screenshot

```

$ btcdeb a9147dcca3d2eb917af831ac29f90de4981bfc62d43e87 0014e47ab4b53c8448cdff890d767cefeb7cd16b98bc
btcdeb 5.0.24 -- type 'btcdeb -h' for start up options
LOG: signing segwit taproot
notice: btcdeb has gotten quieter; use --verbose if necessary (this message is temporary)
6 op script loaded. type 'help' for usage information
script | stack
-----|-----
OP_HASH160 | 0014e47ab4b53c8448cdff890d767cefeb7cd16b98bc
7dcca3d2eb917af831ac29f90de4981bfc62d43e |
OP_EQUAL |
<<< P2SH script >>> |
0 |
e47ab4b53c8448cdff890d767cefeb7cd16b98bc |
#0000 OP_HASH160 |
btcdeb> step
      <> POP stack
      <> PUSH stack 7dcca3d2eb917af831ac29f90de4981bfc62d43e
script | stack
-----|-----
7dcca3d2eb917af831ac29f90de4981bfc62d43e | 7dcca3d2eb917af831ac29f90de4981bfc62d43e
OP_EQUAL |
<<< P2SH script >>> |
0 |
e47ab4b53c8448cdff890d767cefeb7cd16b98bc |
#0001 7dcca3d2eb917af831ac29f90de4981bfc62d43e |
btcdeb> step
      <> PUSH stack 7dcca3d2eb917af831ac29f90de4981bfc62d43e
script | stack
-----|-----
OP_EQUAL | 7dcca3d2eb917af831ac29f90de4981bfc62d43e
<<< P2SH script >>> | 7dcca3d2eb917af831ac29f90de4981bfc62d43e
0 |
e47ab4b53c8448cdff890d767cefeb7cd16b98bc |
#0002 OP_EQUAL |
btcdeb> step
      <> POP stack
      <> POP stack
      <> PUSH stack 01
script | stack
-----|-----
<<< P2SH script >>> | 01
0 |
e47ab4b53c8448cdff890d767cefeb7cd16b98bc |
<<< P2SH script >>> |
btcdeb> step
      <> POP stack
script | stack
-----|-----
0 |
e47ab4b53c8448cdff890d767cefeb7cd16b98bc |
#0004 0 |
btcdeb> step
      <> PUSH stack
script | stack
-----|-----
e47ab4b53c8448cdff890d767cefeb7cd16b98bc | 0x
#0005 e47ab4b53c8448cdff890d767cefeb7cd16b98bc |
btcdeb> step
      <> PUSH stack e47ab4b53c8448cdff890d767cefeb7cd16b98bc
script | stack
-----|-----
e47ab4b53c8448cdff890d767cefeb7cd16b98bc | 0x
btcdeb> step
script | stack
-----|-----

```

Figure 6: btcdeb execution of SegWit A' → B' transaction

4.2 Transaction B' → C'

The next transaction spends the output created in the previous transaction.

Workflow

1. The program retrieves **unspent outputs** using the `listunspent` RPC command.
2. The UTXO generated in the **A' → B' transaction** is identified using the transaction ID:

c2856b601b41c97887117af331d2683255e5da9bfac3165d0ef6786b7b9e55d7

3. This UTXO becomes the **input for the next transaction**.
4. A raw transaction is created sending **2.5 BTC from Address B' to Address C'**.
5. The remaining balance is returned to **B' as change**.
6. The transaction is signed using the wallet private key.
7. The signed transaction is broadcast to the network.
8. A block is mined to confirm the transaction.

Transaction ID

The transaction ID generated for **B' → C'** is:

52c2b0b2c94a7da9e5ef736766e7e2472c01fa5d4a6242b24f1b9aa232cca3c0

Relationship Between the Two SegWit Transactions

The **A' → B' transaction** created a **UTXO assigned to Address B'**. This output is referenced using the transaction ID:

c2856b601b41c97887117af331d2683255e5da9bfac3165d0ef
6786b7b9e55d7

When the **B' → C' transaction** is created, this UTXO is selected as the **input**, proving that Address B' has the right to spend the coins.

Thus, the output of the **A' → B'** transaction becomes the **input of the B' → C'** transaction, demonstrating how SegWit transactions also follow the **UTXO chaining mechanism** used in Bitcoin.

Screenshot

```
[Running] python -u "c:\Users\rpchan\bitcoin_lab\part2_segwit_BC.py"
Connected to Bitcoin Core
Connected to wallet
P2SH-SegWit Address C': 2N3MZHT5iz2mykFUCNrGj1h6goxaQmpYfeq
Selected UTXO: {'txid': 'c2856b60b141c97887117af331d268325e5da9bfac3165d0ef6786b7b9e55d7', 'vout': 0, 'address': '2N7sXpAHa2KwCiDDeg2X3PLuFtJmP3umy6W', 'label': 'B_prime', 'redeemScript': '001455682083f0b5d80ef2f7015ba09bb0e87762cfbf', 'scriptPubKey': 'a914a06f9d946c91fc2f4e70f437f92aa7b5bfe23cc687', 'amount': Decimal('3.00000000'), 'confirmations': 1, 'spendable': True, 'solvable': True, 'desc': 'sh(wpkh([aabbf3a2/49h/1h/0h/0/36]03fc12f6e1b434b3a735c74e7f62ec76c676670147eb5f12243fd970adad5bdc))#kpwf6h3d', 'parent_descs': ['sh(wpkh([aabbf3a2/49h/1h/0h]tpubDr1t8AuV9RNBmRmHxVLCrQhuFeTgJC8QWogLgKzuwCBCreyPSDiiFZ5C51F2XmZPnFrPt2mf7jgdJhCYMb1NpLyowVNLjCECEa5DvZE22W/0*))#u0z2v3qa7'], 'safe': True}

Raw Transaction (B' -> C'):
0200000001d759e7b6b78f60e5d16c3fa9bdae5553268d231f37a118778c9411b606b85c2000000000fdfffff0280b2e60e000000017a9146ee3de061d6ecdffb2100c462a622bc923d0b2438770c9fa02000000017a914a06f9d946c91fc2f4e70f437f92aa7b5bfe23cc68700000000
438770c9fa02000000017a914a06f9d946c91fc2f4e70f437f92aa7b5bfe23cc68700000000

Decoded Raw Transaction:
{'txid': 'fa5ffa92225a2614289231228d97960a74bbd2a54572d5dccc8ae62efa72f6a7', 'hash': 'fa5ffa92225a2614289231228d97960a74bbd2a54572d5dccc8ae62efa72f6a7', 'version': 2, 'size': 115, 'weight': 115, 'locktime': 0, 'vin': [{'txid': 'c2856b60b141c97887117af331d268325e5da9bfac3165d0ef6786b7b9e55d7', 'vout': 0, 'scriptSig': {'asm': '', 'hex': ''}, 'sequence': 4294967293}], 'vout': [{'value': Decimal('2.50000000'), 'n': 0, 'scriptPubKey': {'asm': 'OP_HASH160 6ee3de061d6ecdffb2100c462a622bc923d0b243 OP_EQUAL', 'desc': 'addr(2N3MZHT5iz2mykFUCNrGj1h6goxaQmpYfeq)#aw1s7x50', 'hex': 'a9146ee3de061d6ecdffb2100c462a622bc923d0b24387', 'address': '2N3MZHT5iz2mykFUCNrGj1h6goxaQmpYfeq', 'type': 'scripthash'}}, {'value': Decimal('0.49990000'), 'n': 1, 'scriptPubKey': {'asm': 'OP_HASH160 a06f9d946c91fc2f4e70f437f92aa7b5bfe23cc6 OP_EQUAL', 'desc': 'addr(2N7sXpAHa2KwCiDDeg2X3PLuFtJmP3umy6W)#5yl14lvxj', 'hex': 'a914a06f9d946c91fc2f4e70f437f92aa7b5bfe23cc687', 'address': '2N7sXpAHa2KwCiDDeg2X3PLuFtJmP3umy6W', 'type': 'scripthash'}}]}

Signed Transaction:
0200000001d759e7b6b78f60e5d16c3fa9bdae5553268d231f37a118778c9411b606b85c200000001716001455682083f0b5d80ef2f7015ba09bb0e87762cfbfdf0280b2e60e000000017a9146ee3de061d6ecdffb2100c462a622bc923d0b2438770c9fa0200000000017a914a06f9d946c91fc2f4e70f437f92aa7b5bfe23cc687024730440220168acdbcc29a13b99d27b26b8bc4ad732f8c9a3d1be37c95c5697ccc6ba48a27022016a59ac36e4b84f1d821344b128ae224ac8f87d7b168f40193e5a07082de734012103fc12f6e1b434b3a735c74e7f62ec76c676670147eb5f12243fd970adad5bdc00000000

B' -> C' Transaction ID: 52c2b0b2c94a7da9e5ef736766e7e2472c01fa5d4a624b24fb1b9aa232cca3c0
Block mined to confirm B' -> C'

B' -> C' Transaction ID: 52c2b0b2c94a7da9e5ef736766e7e2472c01fa5d4a624b24fb1b9aa232cca3c0
Block mined to confirm B' -> C'

Final Decoded Transaction:
{'txid': '52c2b0b2c94a7da9e5ef736766e7e2472c01fa5d4a624b24fb1b9aa232cca3c0', 'hash': '65e0784a8168cd8e63acb31bf70e81b86f14d3b6feef990cead631c4714076e', 'version': 2, 'size': 247, 'vsize': 166, 'weight': 661, 'locktime': 0, 'vin': [{'txid': 'c2856b60b141c97887117af331d268325e5da9bfac3165d0ef6786b7b9e55d7', 'vout': 0, 'scriptSig': {'asm': '001455682083f0b5d80ef2f7015ba09bb0e87762cfbf', 'hex': '16001455682083f0b5d80ef2f7015ba09bb0e87762cfbf', 'txinWitness': ['30440220168acdbcc29a13b99d27b26b8bc4ad732f8c9a3d1be37c95c5697ccc6ba48a27022016a59ac36e4b84f1d821344b128ae224ac8f87d7b168f40193e5a07082de73401', '03fc12f6e1b434b3a735c74e7f62ec76c676670147eb5f12243fd970adad5bdc'], 'sequence': 4294967293}], 'vout': [{'value': Decimal('2.50000000'), 'n': 0, 'scriptPubKey': {'asm': 'OP_HASH160 6ee3de061d6ecdffb2100c462a622bc923d0b243 OP_EQUAL', 'desc': 'addr(2N3MZHT5iz2mykFUCNrGj1h6goxaQmpYfeq)#aw1s7x50', 'hex': 'a9146ee3de061d6ecdffb2100c462a622bc923d0b24387', 'address': '2N3MZHT5iz2mykFUCNrGj1h6goxaQmpYfeq', 'type': 'scripthash'}}, {'value': Decimal('0.49990000'), 'n': 1, 'scriptPubKey': {'asm': 'OP_HASH160 a06f9d946c91fc2f4e70f437f92aa7b5bfe23cc6 OP_EQUAL', 'desc': 'scripthash', 'hex': 'a914a06f9d946c91fc2f4e70f437f92aa7b5bfe23cc687', 'address': '2N7sXpAHa2KwCiDDeg2X3PLuFtJmP3umy6W', 'type': 'scripthash'}}]}

scriptPubKey:
{'asm': 'OP_HASH160 6ee3de061d6ecdffb2100c462a622bc923d0b243 OP_EQUAL', 'desc': 'addr(2N3MZHT5iz2mykFUCNrGj1h6goxaQmpYfeq)#aw1s7x50', 'hex': 'a9146ee3de061d6ecdffb2100c462a622bc923d0b24387', 'address': '2N3MZHT5iz2mykFUCNrGj1h6goxaQmpYfeq', 'type': 'scripthash'}
```

Figure 7: Execution output of SegWit $B' \rightarrow C'$ transaction

Transaction ID

e267232dbde285e21156b757e19a0e84e97d445c32acf5e6b514b78a136fc14e

This transaction spends the output from the **A' → B'** **transaction** and transfers coins from **Address B' to Address C'**.

Locking Script (scriptPubKey)

The locking script for the output sending coins to Address C' is:

OP_HASH160 5bfd92da8faf6da4bce6b6bedb986ab78cdb4f56
OP_EQUAL

This script locks the coins to the **P2SH-SegWit address of C'**.

Unlocking Script (scriptSig)

The unlocking script contains the redeem script corresponding to the SegWit address:

0014398b0abeb3597560e72203a1431108e04e4c6e82

This redeem script indicates the **SegWit witness program used for the transaction**.

Witness Data

The witness field contains the following data used to validate the signature:

<signature>

<public key>

These values prove ownership of the private key corresponding to Address B'.

Script Analysis

SegWit validation occurs in two stages:

Stage 1 — P2SH validation

OP_HASH160 <scriptHash> OP_EQUAL

Stage 2 — Witness validation

OP_DUP OP_HASH160 <pubKeyHash> OP_EQUALVERIFY
OP_CHECKSIG

Screenshot

```
└─$ btcdeb a914a06f9d946c91fc2f4e70f437f92aa7b5bfe23cc687 001455682083f0b5d80ef2f7015ba09bb0e87762cffb
btcdeb 5.0.24 -- type `btcdeb -h` for start up options
LOG: signing segwit taproot
notice: btcdeb has gotten quieter; use --verbose if necessary (this message is temporary)
6 op script loaded. type `help` for usage information
script
-----+----- stack
OP_HASH160 | 001455682083f0b5d80ef2f7015ba09bb0e87762cffb
a06f9d946c91fc2f4e70f437f92aa7b5bfe23cc6 |
OP_EQUAL |
<<< P2SH script >>> |
0 |
55682083f0b5d80ef2f7015ba09bb0e87762cffb |
#0000 OP_HASH160 |
btcdeb> step
      <> POP stack
      <> PUSH stack a06f9d946c91fc2f4e70f437f92aa7b5bfe23cc6
script
-----+----- stack
a06f9d946c91fc2f4e70f437f92aa7b5bfe23cc6 | a06f9d946c91fc2f4e70f437f92aa7b5bfe23cc6
OP_EQUAL |
<<< P2SH script >>> |
0 |
55682083f0b5d80ef2f7015ba09bb0e87762cffb |
#0001 a06f9d946c91fc2f4e70f437f92aa7b5bfe23cc6 |
btcdeb> step
      <> PUSH stack a06f9d946c91fc2f4e70f437f92aa7b5bfe23cc6
script
-----+----- stack
OP_EQUAL | a06f9d946c91fc2f4e70f437f92aa7b5bfe23cc6
<<< P2SH script >>> | a06f9d946c91fc2f4e70f437f92aa7b5bfe23cc6
0 |
55682083f0b5d80ef2f7015ba09bb0e87762cffb |
#0002 OP_EQUAL |
btcdeb> step
      <> POP stack
      <> POP stack
btcdeb> step
      <> POP stack
      <> POP stack
      <> PUSH stack 01
script
-----+----- stack
<<< P2SH script >>> | 01
0 |
55682083f0b5d80ef2f7015ba09bb0e87762cffb |
<<< P2SH script >>> |
btcdeb> step
      <> POP stack
script
-----+----- stack
0 |
55682083f0b5d80ef2f7015ba09bb0e87762cffb |
#0004 0 |
btcdeb> step
      <> PUSH stack
script
-----+----- stack
55682083f0b5d80ef2f7015ba09bb0e87762cffb | 0x
#0005 55682083f0b5d80ef2f7015ba09bb0e87762cffb |
btcdeb> step
      <> PUSH stack 55682083f0b5d80ef2f7015ba09bb0e87762cffb
script
-----+----- stack
| 55682083f0b5d80ef2f7015ba09bb0e87762cffb
| 0x
btcdeb> step
script
-----+----- stack
| 55682083f0b5d80ef2f7015ba09bb0e87762cffb
| 0x
btcdeb> step
at end of script
btcdeb> 
```

Figure 8: btcdeb debugging of SegWit transaction

Structure of Challenge and Response Scripts in SegWit Transactions

Bitcoin transactions use a **challenge–response mechanism** to verify ownership of funds. This mechanism is implemented using **Bitcoin Script**, where each transaction output contains a **locking script (challenge script)** and the spending transaction provides an **unlocking script (response script)**.

In this experiment, the SegWit transactions $A' \rightarrow B'$ and $B' \rightarrow C'$ use the **P2SH-P2WPKH (Pay-to-Script-Hash wrapped Pay-to-Witness-Public-Key-Hash)** structure.

Challenge Script (Locking Script)

The locking script defines the conditions required to spend the coins. In the SegWit transactions, the locking script used is:

OP_HASH160 <ScriptHash> OP_EQUAL

Example from the decoded transaction:

OP_HASH160

72b603fe85385210b55128e61f2cb8d1a85a1d58 OP_EQUAL

This script locks the coins to a **P2SH address**. The hash inside the script corresponds to the **redeem script** associated with the SegWit address.

The locking script acts as the **challenge**, requiring the spender to provide the correct redeem script and valid signature.

Response Script (Unlocking Script)

The unlocking script provides the data needed to satisfy the locking script.

For P2SH-SegWit transactions, the unlocking script contains the **redeem script**, which indicates that the transaction uses a SegWit witness program.

Example redeem script:

001461a42d4ebf57b21510b1624e7bfe0fdfe4f8e9a

This redeem script represents a **P2WPKH witness program**.

The actual signature and public key used for validation are stored in the **witness field** of the transaction:

<signature>

<public key>

Transaction Validation Process

When a SegWit transaction is validated, the Bitcoin node executes the scripts in multiple stages.

Step 1: Validate the P2SH Locking Script

The unlocking script provides the redeem script. The Bitcoin node hashes the redeem script and compares it with the hash stored in the locking script.

OP_HASH160 <ScriptHash> OP_EQUAL

If the hashes match, the redeem script is accepted.

Step 2: Execute the Witness Program

The redeem script reveals the SegWit witness program:

```
OP_DUP OP_HASH160 <PubKeyHash> OP_EQUALVERIFY  
OP_CHECKSIG
```

The witness data provides the signature and public key.

Execution steps:

1. The public key is duplicated using **OP_DUP**.
2. The public key is hashed using **OP_HASH160**.
3. The hash is compared with the expected public key hash using **OP_EQUALVERIFY**.
4. **OP_CHECKSIG** verifies the digital signature using the public key.

Final Validation

If all operations succeed and the signature is valid, the script execution leaves **TRUE on the stack**.

TRUE

This confirms that the transaction satisfies the conditions defined in the locking script, and the transaction is considered valid.

5 Comparison of Legacy and SegWit Transactions

Feature	Legacy (P2PKH)	SegWit (P2SH-P2WPKH)
Signature location	scriptSig	witness
Transaction malleability	possible	fixed
Transaction weight	higher	lower
Block efficiency	lower	higher

Size Comparison

Transaction Type	Size	vSize	Weight
Legacy	225 bytes	225	900
SegWit	247 bytes	166	661

Although the raw size appears similar, SegWit has a **smaller virtual size and lower weight**, making it more efficient.

Transaction Size Comparison

To analyze the efficiency of SegWit, the sizes of the **legacy P2PKH transactions** and **SegWit P2SH-P2WPKH transactions** were examined using the decoded transaction outputs.

Transaction size metrics include:

- **size** → raw byte size of the transaction
- **vsize (virtual size)** → effective size used for fee calculation
- **weight** → SegWit weight metric

Size of Legacy Transactions (P2PKH)

For the legacy transactions **A → B** and **B → C**, the decoded transaction output shows the following size:

Trans action	TXID	Siz e	vSi ze	Wei ght
		22	22	
A → B	d1431f1c336849bf4b642376d95cf8d76cb 2f179b8a549a056d4ffe41e38cea6	5 by tes	5 vby tes	900
		22	22	
B → C	a1f2e9da4c2f952a633b73004e77474be6 418a68ba6e55cff58c7356bf5616a4	5 by tes	5 vby tes	900

In legacy transactions, the **signature data is stored inside the transaction input (scriptSig)**.

This increases the transaction size because the signature and public key are included directly in the main transaction structure.

Size of SegWit Transactions (P2SH-P2WPKH)

For the SegWit transactions $A' \rightarrow B'$ and $B' \rightarrow C'$, the decoded transaction shows:

Transaction	TXID	Size	Virtual Size (vsize)	Witnesses
$A' \rightarrow B'$	013c4e28466e0f220703273c4e8c417d152ae3550e1b82f687b1a05572bfbb51	247 bytes	166 vbytes	661
$B' \rightarrow C'$	e267232dbde285e21156b757e19a0e84e97d445c32acf5e6b514b78a136fc14e	247 bytes	166 vbytes	661

Although the raw size of the SegWit transaction appears slightly larger, the **virtual size (vsize)** is significantly smaller because the signature data is stored separately in the **witness field**.

Analysis

The comparison shows that:

- Legacy P2PKH transactions have a **virtual size of 225 vbytes**.
- SegWit P2SH-P2WPKH transactions have a **virtual size of 166 vbytes**.

Thus, SegWit transactions reduce the effective transaction size used for fee calculation.

This improvement occurs because **SegWit separates the witness data (signature and public key) from the main transaction data**, reducing the amount of data counted toward block size limits.

The results demonstrate that **SegWit transactions are more space-efficient than legacy P2PKH transactions**.

Even though the raw transaction size may appear larger, the **virtual size is smaller**, which reduces transaction fees and increases the number of transactions that can fit inside a block.

Comparison of Script Structures: P2PKH vs P2SH-P2WPKH

Bitcoin transactions use a **challenge–response mechanism** implemented through locking scripts (challenge) and unlocking scripts (response). In this experiment, two different script structures were analyzed:

- **P2PKH (Pay-to-Public-Key-Hash)** – Legacy transactions
- **P2SH-P2WPKH (Pay-to-Script-Hash wrapped SegWit)** – SegWit transactions

These two transaction types differ in both **script structure** and **transaction size metrics (weight and virtual bytes)**.

1. Script Structure

P2PKH (Legacy Transactions)

In legacy transactions, the locking script specifies the public key hash of the recipient.

Locking Script (Challenge)

```
OP_DUP OP_HASH160 <PubKeyHash> OP_EQUALVERIFY  
OP_CHECKSIG
```

This script requires the spender to provide a valid signature and public key corresponding to the public key hash.

Unlocking Script (Response)

```
<signature> <public key>
```

During validation, the unlocking script pushes the signature and public key onto the stack, and the locking script verifies them using the defined opcodes.

Thus, both the **signature and public key are stored in the scriptSig**, which increases the transaction size.

P2SH-P2WPKH (SegWit Transactions)

In SegWit transactions, the locking script uses a **P2SH structure**, while the signature and public key are stored in the **witness field**.

Locking Script (Challenge)

```
OP_HASH160 <ScriptHash> OP_EQUAL
```

This script locks the coins to a redeem script hash.

Unlocking Script (Response)

The unlocking script contains the redeem script:

0014 <PubKeyHash>

This redeem script indicates a **P2WPKH witness program**.

Witness Data

The actual signature verification occurs in the witness field:

<signature>

<public key>

Thus, SegWit separates the signature data from the main transaction body.

2. Transaction Size Comparison

The size of the transactions was analyzed using the decoded outputs.

Transaction Type	Script Location	Size	vSize	Weight
P2PKH (Legacy)	scriptSig	225 bytes	225 vbytes	900
P2SH-P2WPKH (SegWit)	witness	247 bytes	166 vbytes	661

3. Key Differences

Feature	P2PKH (Legacy)	P2SH-P2WPKH (SegWit)
Locking Script	OP_DUP OP_HASH160 ... OP_CHECKSIG	OP_HASH160 ... OP_EQUAL
Unlocking Script	signature + public key	redeem script
Signature Location	scriptSig	witness field
Transaction Size	larger	smaller (in vbytes)
Weight	higher	lower

4. Analysis

Legacy transactions include signature data directly inside the **scriptSig**, which increases the size of the transaction. In contrast, SegWit transactions move the signature and public key to the **witness field**, which reduces the effective size used for block limits.

As a result, SegWit transactions have a **smaller virtual size (vbytes)** and **lower weight**, allowing more transactions to fit inside a block.

Why SegWit Transactions are Smaller and Their Benefits

SegWit (Segregated Witness) is a protocol upgrade in Bitcoin that changes how transaction data is stored and validated. It separates the **signature data (witness data)** from the main transaction structure, which reduces the effective transaction size used for block limits.

Why SegWit Transactions are Smaller

In legacy **P2PKH transactions**, the digital signature and public key are stored inside the **scriptSig field** of the transaction input. Since signatures are large (usually around 70–72 bytes), they significantly increase the transaction size.

For example, the legacy transactions in this experiment had a **virtual size of 225 vbytes**.

In SegWit transactions (**P2SH-P2WPKH**), the signature and public key are moved to a separate structure called the **witness field**. This witness data is not counted fully toward the transaction size when calculating the block limit.

Instead, SegWit uses a **weight-based accounting system**, where witness data is given a lower weight compared to the main transaction data. As a result, the **virtual size (vbytes)** of SegWit transactions becomes smaller.

In this experiment:

- Legacy transaction size ≈ **225 vbytes**
- SegWit transaction size ≈ **166 vbytes**

Thus, SegWit transactions appear slightly larger in raw bytes but are **smaller in effective size** due to the witness discount.

Benefits of SegWit Transactions

SegWit provides several important improvements to the Bitcoin network.

1. Reduced Transaction Size

Because witness data receives a size discount, SegWit transactions require fewer **virtual bytes**, allowing more transactions to fit into a single block.

2. Lower Transaction Fees

Transaction fees are calculated based on **vbytes**. Since SegWit transactions have smaller virtual sizes, they generally result in **lower transaction fees** compared to legacy transactions.

3. Increased Block Capacity

SegWit introduced the concept of **block weight**, allowing blocks to include more transactions without increasing the original 1 MB block size limit. This improves the overall throughput of the Bitcoin network.

4. Fixes Transaction Malleability

SegWit eliminates the issue of **transaction malleability**, where transaction IDs could be modified before confirmation. This fix is important for enabling advanced Bitcoin

technologies such as **payment channels and the Lightning Network**.

6 Conclusion

This experiment demonstrated the creation and validation of Bitcoin transactions using both Legacy P2PKH and SegWit P2SH-P2WPKH transaction formats in the Bitcoin Regtest environment. The workflow showed how the output of one transaction becomes the input for the next transaction using the UTXO model, forming a chain of transactions.

The script analysis revealed that both transaction types follow the challenge–response mechanism of Bitcoin Script. In legacy transactions, the locking script specifies the public key hash of the recipient and the unlocking script provides the signature and public key required for validation. In SegWit transactions, the locking script uses a P2SH structure, while the signature and public key are stored in the witness field, separating signature data from the main transaction structure.

The comparison of transaction sizes showed that legacy P2PKH transactions had a virtual size of approximately 225 vbytes, whereas SegWit P2SH-P2WPKH transactions had a smaller virtual size of about 166 vbytes due to the witness discount. Although the raw transaction size may appear larger, SegWit reduces the effective size used for block limits.

Overall, SegWit improves the efficiency of Bitcoin transactions by reducing transaction size, lowering

transaction fees, increasing block capacity, and eliminating transaction malleability. These improvements make SegWit an important upgrade that enhances the scalability and security of the Bitcoin network.

7 GitHub Repository

<https://github.com/Bhavyasri104/CS-216-BlockSmisths-Bitcoin-Transaction-Lab>
